Network Programming – Theory Exam Guide

Scope: POSIX Semaphores → Shared Memory → Socket Programming → TCP client/server → exceptions → concurrent servers → select() → shutdown(). Built from Prof. Anupama Potluri's UNP slides, her past Minor papers, and W. Richard Stevens (UNP Vol. 1 & Vol. 2).

0. How to use this guide

What her past papers show about her style:

- "What happens when…?" questions (child closes fd but parent doesn't; server host crashes; bind is missing).
- Exact names: which errno, which signal, which flag. "ETIMEDOUT vs EHOSTUNREACH", "SIGPIPE", "ECONNRESET", "MSG_PEEK".
- Code snippets where you must predict the output or find the bug (the semaphore-initialised-to-zero question).
- "Give two reasons…", "Differences between X and Y" – so for every pair (mutex/semaphore, close/shutdown, read/recv, MAP_SHARED/MAP_PRIVATE) know 3 crisp differences.
- Full marks went to answers that gave the *mechanism* (what the kernel does), not just the definition. Always explain *why*.

For every system call, be able to state: prototype → each argument → return value on success/failure → important errno values → what the kernel does internally → a typical use.

Honest caveat: this guide is built from the slides plus my knowledge of Stevens, not from a copy of the book in front of me. Chapter numbers are for UNP Vol. 1 (3rd ed.) and Vol. 2 (2nd ed.). Since she expects you to have read Stevens line by line, use this guide as the skeleton and skim the chapters below to fill in any detail.

Stevens reading map

- **UNP Vol. 2 (IPC), Ch. 10** – Posix Semaphores (incl. producer-consumer with semaphores).
- **UNP Vol. 2, Ch. 12–13** – Shared Memory Introduction; Posix Shared Memory (mmap, munmap, msync, shm_open, shm_unlink, ftruncate).

- **UNP Vol. 1, Ch. 3** – Sockets Introduction (sockaddr structures, value-result arguments, byte ordering, byte manipulation functions, inet_* functions).
- **UNP Vol. 1, Ch. 4** – Elementary TCP Sockets (socket, connect, bind, listen, accept, fork, concurrent servers, close, getsockname/getpeername).
- **UNP Vol. 1, Ch. 5** – TCP Client/Server Example (handshake and termination, POSIX signal handling, SIGCHLD and zombies, wait vs waitpid, ECONNABORTED, SIGPIPE, server crash / reboot / shutdown).
- **UNP Vol. 1, Ch. 6** – I/O Multiplexing (five I/O models, select, str_cli, batch input, shutdown, TCP server with select, poll).
- **UNP Vol. 1, Ch. 7** – Socket Options (SO_REUSEADDR, SO_KEEPALIVE etc.) – appears in her Minor 3.

---

1. POSIX Semaphores (UNP Vol. 2, Ch. 10)

1.1 What a semaphore is

A semaphore is a non-negative integer with two atomic operations:

- **wait (P, down):** if value > 0, decrement and continue; if value == 0, block until it becomes > 0, then decrement.
- **post (V, up):** increment the value; if any process/thread is blocked in wait, wake one up.

The key property: the test-and-decrement is atomic in the kernel/library, so two waiters can never both get the last unit.

- **Binary semaphore:** value is only 0 or 1 → used for mutual exclusion (initial value 1).
- **Counting semaphore:** value counts available resources ("one per resource available", as the slide says). When all are in use a process needing one must wait until one is released.
- Stevens contrasts **Posix semaphores** (simple: one counter, sem_open/sem_wait/sem_post) with **System V semaphores** (semget/semop/semctl; operate on *sets* of semaphores, creation and initialisation are two separate non-atomic steps, so they have a race).

1.2 Persistence of IPC objects (asked in her Minor 1)

Persistence = how long an IPC object lives.

- **Process-persistent:** exists until the last process holding it closes it. Examples: pipes, FIFOs (the FIFO *name* stays in the filesystem, but the data is lost once the last process closes it), sockets.
- **Kernel-persistent:** exists until the kernel reboots or the object is explicitly deleted. Value/contents survive even when no process has it open. Examples: System V message queues, semaphores, shared memory; Posix message queues, semaphores, shared memory (until sem_unlink / shm_unlink).
- **Filesystem-persistent:** exists until explicitly deleted and survives a kernel reboot, because it is backed by a file. Examples: a memory-mapped file; memory-based semaphores that live inside a shared region backed by a file (the slide says semaphores are "typically kernel-persistent but can also be file-persistent" in this case).

1.3 Two kinds of Posix semaphores

1. **Named semaphores** – created/opened with `sem_open(name, ...)`. Identified by a name (should begin with a `/` and contain no further slashes, for portability). Unrelated processes can share one by opening the same name. Closed with `sem_close`, removed with `sem_unlink`.
2. **Memory-based (unnamed) semaphores** – a `sem_t` variable that the *caller* allocates (global variable for threads, or inside a shared memory region for processes) and initialises with `sem_init`; removed with `sem_destroy`. No name, no kernel-persistence beyond the memory they live in.

1.4 The function calls in detail

```
#include <semaphore.h>
sem_t *sem_open(const char *name, int oflag, ... /* mode_t mode, unsigned int
value */);
int sem_close(sem_t *sem);
int sem_unlink(const char *name);
int sem_wait(sem_t *sem);
int sem_trywait(sem_t *sem);
int sem_post(sem_t *sem);
int sem_getvalue(sem_t *sem, int *valp);
int sem_init(sem_t *sem, int pshared, unsigned int value);
int sem_destroy(sem_t *sem);
```

sem_open

- `oflag` = 0 → open an existing semaphore (fails with ENOENT if it does not exist).
- `O_CREAT` → create if it does not exist. Now the **mode** (permission bits, like open()) and **value** (initial value) arguments are *required*. If it already exists, mode and value are ignored and the existing one is opened.
- `O_CREAT | O_EXCL` → create; fail with **EEXIST** if it already exists. This is how you make sure *you* initialised it.
- Creation **and** initialisation are one atomic step – this removes the System V race.
- Returns a pointer to `sem_t`, or **SEM_FAILED** on error (not NULL – a classic trap).
- Initial value must be ⩽ SEM_VALUE_MAX (at least 32767).

sem_close – releases the process's reference to the semaphore. It is also done automatically at process exit, whether or not the process called it. It does **not** remove the semaphore and does not change its value.

sem_unlink – removes the *name* immediately (like unlink() on a file). The semaphore itself is destroyed only when every process that has it open has closed it (reference counting). If you never unlink, a named semaphore stays in the kernel even after your program exits – so a rerun with `O_CREAT|O_EXCL` fails with EEXIST. That is why Stevens calls sem_unlink before creating.

sem_wait – if value > 0: decrement and return 0 immediately. If value == 0: block. Can be interrupted by a signal and return -1 with errno = **EINTR** (restart it).

sem_trywait – the non-blocking version: if value == 0 it returns -1 with errno = **EAGAIN** instead of blocking.

sem_post – increment the value and wake up one waiter if any. Stevens points out that sem_post is the only synchronisation function that is **async-signal-safe** (safe to call from a signal handler); mutex unlock and cond_signal are not.

sem_getvalue – returns the current value in `*valp`. It can already be stale when you look at it. If waiters exist, POSIX allows either 0 or a negative number whose magnitude is the number of waiters (Linux returns 0).

sem_init(sem, pshared, value) – `pshared = 0`: shared between threads of one process. `pshared != 0`: shared between processes, but then the `sem_t` must itself be in shared memory. Returns -1 on error (not SEM_FAILED). Initialising an already-initialised semaphore, or destroying one that threads are waiting on, is undefined.

Across fork(): a child inherits the parent's open named semaphores, so both refer to the same semaphore. A memory-based semaphore is shared across fork only if it is in shared memory and pshared != 0.

Compile with `-pthread` (older systems: `-lrt`).

1.5 Semaphores vs Mutexes vs Condition Variables

- **Ownership:** a mutex must be unlocked by the same thread that locked it. A semaphore is usually posted by one process/thread and waited for by another – there is no owner.
- **States:** a mutex is only locked/unlocked and cannot by itself wait for a condition. A semaphore handles both mutual exclusion (binary, initial value 1) *and* waiting for a condition / counting resources.
- **Lost wake-ups:** a condition-variable signal is lost if nobody is waiting at that instant (that is why a predicate and a `while` loop are required). A semaphore post is **never lost** – the value remembers it.
- **Overhead:** a mutex is typically implemented as a different, lighter mechanism than a semaphore, so it is much cheaper. A mutex is a binary semaphore in concept only.
- A condition variable is always paired with a mutex, because the predicate is shared data accessed by several threads.
- Scope: mutexes/condvars are for threads (or processes only with PTHREAD_PROCESS_SHARED in shared memory); semaphores work between threads *or* processes.

1.6 Producer-consumer with semaphores (the slide program)

Shared buffer `buff[MAXITEMS]` (MAXITEMS = 10) and three semaphores:

- `mutex`, initial value **1** – protects the buffer (mutual exclusion).
- `nempty`, initial value **MAXITEMS** – number of empty slots (producer waits on this).
- `nstored`, initial value **0** – number of filled slots (consumer waits on this).

```
/* producer */
sem_wait(&shared.nempty);      /* wait for a free slot           */
sem_wait(&shared.mutex);       /* lock the buffer                */
shared.buff[i % MAXITEMS] = i; /* store the item                 */
sem_post(&shared.mutex);       /* unlock                         */
sem_post(&shared.nstored);     /* one more item stored           */

/* consumer */
sem_wait(&shared.nstored);     /* wait for an item               */
sem_wait(&shared.mutex);
use shared.buff[i % MAXITEMS]
sem_post(&shared.mutex);
sem_post(&shared.nempty);      /* one more empty slot            */
```

Why this order matters. The counting semaphore (nempty / nstored) must be waited on *before* the mutex. If the producer locked the mutex first and the buffer was full, it would block on nempty *while holding the mutex*; the consumer could never get the mutex to free a slot → **deadlock**.

Why three semaphores. The mutex gives mutual exclusion; nempty and nstored give *synchronisation* (producer must not overrun the consumer, consumer must not read an empty slot). Mutual exclusion alone is not enough.

In main: `sem_init(&mutex,0,1); sem_init(&nempty,0,MAXITEMS); sem_init(&nstored,0,0);` then pthread_create producer and consumer, pthread_join both, then `sem_destroy` all three. With exactly one producer and one consumer the mutex is strictly unnecessary (the two never touch the same slot when the counting semaphores are right), but it is required once there are multiple producers or consumers.

1.7 Her Minor 1 question – the "mutex semaphore" code

```
producer() {              consumer() {
  sem_wait(mutex);          sem_wait(mutex);
  count++;                  count--;
  sem_post(mutex);          sem_post(mutex);
}                         }
```

If mutex is initialised to 0: the very first `sem_wait(mutex)` finds the value 0 and blocks. The matching `sem_post` is *after* the wait in the same code path, so nobody can ever post. Both threads block forever → **deadlock**, no output, count never changes.

If mutex is initialised to 1: it works as a mutual-exclusion lock. Only one of the two is in the critical section at a time, so `count++` and `count--` (each is a non-atomic load-modify-store) cannot interleave and no update is lost.

But there is still an issue: nothing coordinates *what* is being produced and consumed. The consumer can run `count--` when nothing was produced (count goes negative / consumes a non-existent item), and the producer can run without bound (buffer overflow). Mutual exclusion is not synchronisation.

Fix: add two counting semaphores – `nempty` (initial N) and `nstored` (initial 0) – and use them as in 1.6: producer does `wait(nempty); wait(mutex); count++; post(mutex); post(nstored)`; consumer does `wait(nstored); wait(mutex); count--; post(mutex); post(nempty)`.

1.8 Quick self-test (semaphores)

- Difference between a named and a memory-based semaphore? What does sem_open return on failure?
- What happens to a named semaphore when the process exits? When you call sem_close? sem_unlink?
- Why is sem_post safe in a signal handler but pthread_cond_signal is not?
- Why can a semaphore post not be lost, but a cond signal can?
- What breaks if the producer's two sem_wait calls are swapped?

---

2. Shared Memory (UNP Vol. 2, Ch. 12–13)

2.1 Why shared memory

Shared memory is the IPC with the **least latency**. With pipes, FIFOs and message queues the data is copied user → kernel → user (two copies, plus a system call for every transfer). With shared memory the processes read and write the *same physical pages* mapped into their address spaces; the kernel is involved only in setting up the mapping. After that, communication is just memory reads and writes.

The price: **the kernel does no synchronisation**. The programmer must provide it explicitly (semaphores, mutexes in shared memory, etc.). Without it two processes incrementing one shared counter lose updates, because `counter++` is load → add → store.

Her Minor 2 Q1 – shared memory vs other IPC (full marks answer):

- Shared memory: fast data access between closely related processes; only a single copy of the data exists; synchronisation must be done explicitly by the programmer.
- Other IPC (pipes, FIFOs, message queues): more latency; two or more copies of the data (user → kernel → user); synchronisation is taken care of by the kernel.

2.2 Two types of Posix shared memory

1. **Memory-mapped files** – `open()` a regular file, then `mmap()` it. The region is backed by the file; changes (MAP_SHARED) are eventually written to the file. Filesystem-persistent.
2. **Shared memory objects** – `shm_open()` a *name* (no regular file needed), then `mmap()` the returned descriptor. Kernel-persistent. This is how *unrelated* processes share memory.

Both are used through mmap. (System V shared memory – shmget/shmat/shmdt/shmctl – is the older third family.)

2.3 mmap and munmap

```
#include <sys/mman.h>
void *mmap(void *start, size_t length, int prot, int flags, int fd, off_t
offset);
int munmap(void *start, size_t length);
int msync(void *addr, size_t length, int flags);
```

Purpose (Minor 2 Q4): mmap maps a file or a shared memory object into the address space of the calling process, so the bytes can be read, written or executed through ordinary memory accesses instead of read()/write() calls.

Arguments:

- `start` – desired adconverdress. Almost always NULL so the kernel chooses. A non-NULL value is only a hint unless MAP_FIXED is given (not portable).
- `length` – number of bytes to map; rounded up to a whole number of pages.
- `prot` – `PROT_READ`, `PROT_WRITE`, `PROT_EXEC`, `PROT_NONE`, ORed together. It must be compatible with how the descriptor was opened (e.g. PROT_WRITE with MAP_SHARED on a descriptor opened O_RDONLY fails with EACCES).
- `flags` – exactly one of `MAP_SHARED` or `MAP_PRIVATE`, plus optionally `MAP_FIXED`.

  - **MAP_SHARED:** modifications are visible to other processes that map the same object, and are carried through to the underlying file/object. This is what you need for IPC.
  - **MAP_PRIVATE:** copy-on-write private copy; modifications are visible only to this process and are *not* written back.
- `fd` – the descriptor of the file or shared memory object. It may be closed right after mmap – the mapping remains.
- `offset` – starting byte within the file; must be a multiple of the page size.

Return value: the starting address of the mapped region, or **MAP_FAILED** (`(void *)-1`) on error – not NULL.

The slide diagram: a file referenced by descriptor `fd`; a portion of it, starting at `offset` for `length` bytes, is mapped into the process's address space; mmap's return value is the address in the process's address space where that portion starts (between low and high memory).

munmap removes the mapping (the process's mappings also vanish at exit). Closing the descriptor does *not* unmap. For MAP_SHARED, the kernel's virtual-memory system updates the file on its own schedule; call **msync** to force it: `MS_SYNC` (write synchronously, wait for completion), `MS_ASYNC` (queue the write, return at once), `MS_INVALIDATE` (invalidate other cached copies).

Faults to know:

- Access outside the mapped region → **SIGSEGV**.
- Access inside the last page but beyond the end of the file is allowed (reads zeros, writes are not saved), but access to a page wholly beyond the end of the file → **SIGBUS**.
- To make a file or shm object bigger than its current size use `ftruncate()` (or lseek past the end and write one byte). *Mapping does not extend the file.*

With fork/exec: a mapping is inherited by the child across fork() – so a MAP_SHARED mapping created before fork is shared between parent and child. exec() discards the mappings.

Uses of mmap: memory-mapped I/O on regular files (no read/write system calls, no extra copy – Stevens' file-copy example), and IPC between processes.

2.4 shm_open and shm_unlink

```
#include <sys/mman.h>
int shm_open(const char *name, int oflag, mode_t mode);
int shm_unlink(const char *name);
```

- `name` – a Posix IPC name, begins with `/` (on Linux it appears under /dev/shm).
- `oflag` – `O_RDONLY` or `O_RDWR`, plus optionally `O_CREAT`, `O_EXCL`, `O_TRUNC`. `mode` is used only with O_CREAT.
- Returns a **descriptor** (not a pointer) or -1.
- A newly created object has **size 0**. You must call `ftruncate(fd, length)` before mmap'ing it; `fstat()` tells you the size of an existing one.
- `shm_unlink` removes the name; the object is destroyed after the last process unmaps it. Until it is unlinked, the object is **kernel-persistent**, which is what the slide's summary says.

Typical sequence. Creator: `shm_open(O_RDWR|O_CREAT|O_EXCL)` → `ftruncate` → `mmap(MAP_SHARED)` → `close(fd)` → use memory. Other process: `shm_open(O_RDWR)` → `fstat` for size → `mmap(MAP_SHARED)` → use memory. When finished: `munmap`, and one process calls `shm_unlink`.

2.5 The slide example – and the bugs in it

The slide opens a shared memory object, maps `MY_SHARED_SIZE` bytes with `PROT_READ|PROT_WRITE`, `MAP_SHARED`, closes fd, runs `*ptr++ = i % 256` for each byte, then calls `shm_unlink(ptr)`.

Bug-spotting points (she likes this):

- `shm_unlink` takes the **name** (`argv[1]`), not the pointer.
- `ptr` has been advanced by the loop and no longer points at the start of the region; use a separate pointer (Stevens uses `p`).
- Without a prior ftruncate, a new object has size 0 and writing would raise SIGBUS.

2.6 Self-test (shared memory)

- MAP_SHARED vs MAP_PRIVATE?
- Does closing the descriptor after mmap remove the mapping?
- What signal if you touch a page beyond the end of the file? Beyond the mapping?

- Why must you call ftruncate after shm_open(O_CREAT)?
- Which two kinds of persistence do the two kinds of Posix shared memory have?

---

3. Socket Programming – Foundations (UNP Vol. 1, Ch. 3)

3.1 What a socket is (Minor 2 Q2)

A socket is an IPC mechanism and endpoint of communication between any two processes, on the same host or on different hosts. A connection is identified by a **5-tuple**: {protocol, source IP, source port, destination IP, destination port}.

3.2 Byte ordering – the 5000 vs 34835 question (Minor 2 Q3)

- **Little-endian:** the low-order byte is stored at the lowest address (Intel x86).
- **Big-endian:** the high-order byte is stored at the lowest address.
- **Network byte order is big-endian.** Host byte order is whatever the CPU uses.
- The port 5000 is 0x1388. Stored big-endian the bytes are `13 88`. A little-endian host that reads those two bytes as a host short without converting sees 0x8813 = **34835**. So one system printed the value converted, the other printed the raw network-order value (or the two machines have different host byte orders and one printed without ntohs).
- Conversion macros/functions: `htons()` / `ntohs()` for 16-bit (ports), `htonl()` / `ntohl()` for 32-bit (IP addresses). h = host, n = network, s = short, l = long.
- Use network byte order for the IP address and port fields of `sockaddr_in`. Her slide says: **always use network byte order for all structures** – i.e. for a struct with a 32-bit seq_no and 16-bit field sent between hosts, use htonl/htons on send and ntohl/ntohs on receive. `sin_family` is **not** converted because it is only used inside the local kernel.

3.3 Network data is a stream of bytes – not strings

Network data is not ASCII-only; it is binary data that can contain 0 bytes. The string functions (`strcpy`, `strlen`, `strcmp`, `strcat`) assume the data ends with a `'\0'` and will stop at the first zero byte or run past the end. Instead use the byte-manipulation functions that take an explicit length:

- BSD: `bzero(ptr, n)`, `bcopy(src, dest, n)`, `bcmp(p1, p2, n)`.
- ANSI C: `memset(ptr, 0, n)`, `memcpy(dest, src, n)`, `memcmp(p1, p2, n)`.
- **Argument order trap:** bcopy takes (src, dest) but memcpy takes (dest, src).

3.4 Socket address structures

```
struct in_addr {
    in_addr_t  s_addr;          /* 32-bit IPv4 address, network byte order */
};
struct sockaddr_in {
    uint8_t         sin_len;    /* BSD only; Linux has no such field
*/
    sa_family_t     sin_family; /* AF_INET
*/
    in_port_t       sin_port;   /* 16-bit port, network byte order
*/
    struct in_addr  sin_addr;   /* 32-bit IPv4 address, network byte order
*/
    char            sin_zero[8];/* unused - set to zero (bzero the struct)
*/
};
```

- `sockaddr_in` is 16 bytes. Always `bzero` it before filling so `sin_zero` is zero.
- The socket functions take a generic `struct sockaddr *` (a family field plus 14 bytes of data), because they must work for IPv4, IPv6, Unix domain, etc. That is why you **cast**: `bind(fd, (struct sockaddr *)&servaddr, sizeof(servaddr))`. (When the API was designed there was no `void *`.) `sockaddr_in6` is the IPv6 one; `sockaddr_storage` is big enough for any.
- `INADDR_ANY` is the wildcard address 0.0.0.0 – accept connections on any local interface. Correct use: `htonl(INADDR_ANY)`.

3.5 Value-result arguments

Functions that return an address from the kernel – `accept`, `recvfrom`, `getsockname`, `getpeername` – take the length as a **pointer**. On the call it tells the kernel how much space you provided; on return it holds how many bytes the kernel actually stored. That is why the slide writes `socklen_t len = sizeof(cliaddr);` *before* `accept` and resets it each time through a loop. Functions that pass an address *into* the kernel – `bind`, `connect`, `sendto` – take the length by **value**.

3.6 Address conversion functions

```
int        inet_aton(const char *cp, struct in_addr *inp);
in_addr_t  inet_addr(const char *cp);
char      *inet_ntoa(struct in_addr in);
```

- `inet_aton` – dotted decimal string → binary (network order) in `*inp`. Returns **1 if valid, 0 if invalid** (the opposite of most calls).
- `inet_addr` – same conversion, returns the address; on error returns `INADDR_NONE` (all ones), which is the same as the legitimate broadcast address 255.255.255.255 – so it is deprecated.
- `inet_ntoa` – binary → dotted decimal. Takes the struct by value and returns a pointer to a **static buffer**, so it is not re-entrant / thread-safe.
- Modern, protocol-independent replacements: `inet_pton(af, strptr, addrptr)` (returns 1 OK, 0 bad format, -1 bad family) and `inet_ntop(af, addrptr, strptr, len)` (returns the string pointer or NULL). `INET_ADDRSTRLEN` = 16, `INET6_ADDRSTRLEN` = 46. The p and n stand for presentation and numeric.

---

4. TCP Client and Server (UNP Vol. 1, Ch. 4)

4.1 The call sequence

Server: `socket` → `bind` → `listen` → `accept` (blocks) → `read` / `write` → `close`.

Client: `socket` → `connect` → `write` / `read` → `close`.

```
int socket(int family, int type, int protocol);   /* AF_INET, SOCK_STREAM, 0
-> returns sockfd or -1 */
int bind(int sockfd, const struct sockaddr *myaddr, socklen_t addrlen);
int listen(int sockfd, int backlog);
int accept(int sockfd, struct sockaddr *cliaddr, socklen_t *addrlen);   /*
value-result */
int connect(int sockfd, const struct sockaddr *servaddr, socklen_t addrlen);
int close(int sockfd);
```

- **socket()** – returns a small non-negative descriptor. `type` = SOCK_STREAM (TCP) or SOCK_DGRAM (UDP). Protocol 0 means the default for that type. Nothing is on the network yet.

- **bind()** – assigns a local protocol address (IP + port) to the socket. A server binds its well-known port. A client normally does **not** bind – the kernel picks an **ephemeral port** and the source IP (from the outgoing interface) at connect() time. Bind with port 0 means "kernel, choose an ephemeral port". Error EADDRINUSE if the port is already in use (see SO_REUSEADDR). Ports below 1024 are reserved and need privilege (that is why your lab used ports above 1024).
- **listen()** – converts an unconnected socket into a **passive** listening socket: the kernel will accept incoming SYNs on it. The second argument, backlog, bounds the queues of connections the kernel keeps for this socket (next section). It must be called after bind and before accept.
- **accept()** – takes the first connection off the front of the *completed* queue and returns a brand-new descriptor, the **connected socket**. If the queue is empty it blocks. Returns -1 and errno on failure. The client's address comes back through the value-result arguments.
- **connect()** – starts the TCP three-way handshake. It returns only when the connection is established or an error occurs.
- **close()** – see 4.4.

Listening socket vs connected socket. The server has one *listening* socket for its whole life (listenfd) and, per client, a separate *connected* socket (connfd) returned by accept. Both have the same local port. The connection is identified by the full 5-tuple, so many connected sockets with the same local port can coexist.

getsockname / getpeername. `getsockname` returns the local IP and port of a socket (useful to learn the ephemeral port after connect, or the local IP after accept when the server bound to INADDR_ANY). `getpeername` returns the foreign address.

4.2 The three-way handshake and the two queues (slide: "Incomplete and Complete Queues")

1. Client calls connect → sends **SYN** (carries client's initial sequence number).
2. Server replies **SYN + ACK**.
3. Client replies **ACK** → connect() returns on the client. The server's connection is now established.

For each listening socket the kernel maintains two queues:

- **Incomplete connection queue (SYN_RCVD):** an entry for every SYN received, waiting for the third packet of the handshake. An entry stays here for about one round-trip time to the client.
- **Completed connection queue (ESTABLISHED):** an entry for every connection whose handshake has finished but which the application has not yet accept()ed.

accept() just takes an entry from the front of the completed queue. If the queue is empty the process sleeps. The **backlog** argument bounds the queue sizes (historically the sum of both; Linux uses it for the completed queue). If the queues are full, new SYNs are ignored and the client's TCP retransmits, which is why a full backlog shows up as slow connects rather than errors. Importantly, the **handshake is completed by the kernel**, so a client can connect() successfully and even send data before the server has called accept().

TCP termination in brief: the side that calls close first sends **FIN**; the peer ACKs it, and when the peer closes it sends its own FIN, which is ACKed. The side that closed first ends up in **TIME_WAIT** (2 × MSL) – which is the root of the EADDRINUSE problem on restarting a server.

4.3 INADDR_ANY vs a specific IP in bind() (Minor 2 Q10)

A multihomed host has IP addresses 10.5.0.63 and 14.89.76.104.

- **bind(INADDR_ANY):** the server's local IP is the *wildcard* 0.0.0.0 – it will accept a connection arriving on **any** of its interfaces. The kernel fills in the real local IP address when the connection arrives: it takes it from the **destination IP of the client's SYN**. So for a client that connected to 10.5.0.63, that connected socket's local address is 10.5.0.63; `getsockname` on the connected socket tells you which one.
- **bind(10.5.0.63):** the server will accept only connections whose destination IP is 10.5.0.63. A client connecting to 14.89.76.104 gets no service (RST / ECONNREFUSED).

The typical reason for binding a specific address is to run different servers (or different instances) on the same port number but different IPs.

4.4 close() and reference counts (Minor 3 Q3)

- `close()` on a descriptor decrements the socket's **reference count**. Only when the count reaches 0 does the kernel start TCP termination (sends FIN) and release the socket. It terminates **both** directions at once.
- After fork() the child has copies of every descriptor, so each socket's count is **2**.

- **Question:** a concurrent server's parent does not close connfd, the child does. **Answer:** the child's close() only drops the count from 2 to 1; no FIN is sent, so the **connection stays open** and the client sees no EOF. And the parent leaks a descriptor on every client, so it eventually runs out of descriptors (EMFILE). That is why the parent *must* close connfd and the child *must* close listenfd.

---

5. Concurrent Servers and Zombies (UNP Vol. 1, Ch. 4–5)

5.1 The fork-per-client pattern (slide)

```
listenfd = Socket(...); bind(...); listen(listenfd, LISTENQ);
for ( ; ; ) {
    connfd = Accept(listenfd, ...);
    if ( (pid = Fork()) == 0) {   /* child */
        close(listenfd);          /* child does not need the listening socket
*/
        doit(connfd);
        close(connfd);
        exit(0);
    }
    close(connfd);                /* parent does not need the connected
socket */
}
```

Why it works: after fork, parent and child share both descriptors. The parent goes straight back to accept() for the next client; the child serves this one. Each closes the descriptor it does not use so the reference counts drop correctly. A simple iterative server (no fork) serves only one client at a time and everybody else waits in the backlog.

5.2 Zombies (slide "Process Control – Summary")

- When a child terminates it becomes a **zombie**: the kernel keeps its exit status, resource usage and PID until the parent calls `wait` / `waitpid`. A zombie uses a process-table slot.
- A long-running concurrent server that never waits will eventually run out of process-table entries – so a concurrent server must handle zombies.
- If the **parent dies first**, the child becomes an orphan and is adopted by init (PID 1), which waits for it.

- Solution: establish a handler for **SIGCHLD** (which the kernel sends to the parent when a child terminates) that calls `wait` / `waitpid`.

```
void sig_chld(int signo) {
    pid_t pid; int stat;
    while ( (pid = waitpid(-1, &stat, WNOHANG)) > 0)
        printf("child %d terminated\n", pid);
}
```

wait vs waitpid (the question she likes): `wait` blocks until *any one* child terminates and cannot be told to return immediately. `waitpid(pid, &status, options)` lets you choose the child (pid = -1 means any) and supports `WNOHANG` (return 0 at once if no child has terminated).

Why a loop with waitpid(WNOHANG) and not a single wait()? Unix signals are **not queued**. If five children terminate at nearly the same time, the handler may run only once (SIGCHLD stays pending as a single bit). A single wait() would reap one child and leave four zombies. Looping with WNOHANG reaps all that are ready and never blocks.

Interrupted system calls: SIGCHLD can arrive while the parent is blocked in `accept()`. accept then returns -1 with **EINTR** (on a "slow system call"). You must handle it: either establish the handler with SA_RESTART or test `if (errno == EINTR) continue;`. Other errors from accept must be treated as fatal.

---

6. Exceptional Conditions in TCP (slide: "Socket Function Calls – Data Structures and Exceptions")

6.1 ECONNABORTED – connection aborted before accept returns

The three-way handshake completes (connection sits on the completed queue), then the client sends **RST** (it aborts, e.g. the user kills the client) *before* the server calls accept(). Behaviour depends on the implementation: Berkeley kernels remove the connection from the queue and never tell the application; others (SVR4) make accept() return an error (**ECONNABORTED**, or EPROTO on SVR4). The server should treat ECONNABORTED (and EPROTO) as non-fatal, ignore it and call accept() again.

6.2 Server process terminates prematurely

The server child is killed while the client is connected:

- The server's kernel closes the descriptors and sends **FIN** to the client. The client TCP replies ACK, but the client application is blocked on something else (e.g. fgets on stdin), so it does not notice the FIN.
- When the client next writes, the server TCP, whose socket no longer exists, replies with **RST**.
- A read() after the RST returns **ECONNRESET**; if the FIN was already received it returns 0 (EOF) first. A **write on a socket that has already received an RST** generates **SIGPIPE** (default action: terminate the process). If SIGPIPE is ignored or caught, write returns **EPIPE**.
- Lesson from the slide: a client blocked on stdin cannot notice the server closed. It needs to wait on **both** stdin and the socket – which is exactly what select() is for (section 7).

6.3 Server host crashes

The server *host* dies without sending anything (no FIN, no RST). The client sends data and gets no ACK, so TCP retransmits (Berkeley-derived: about 9 minutes, 12 retries with exponential backoff). Then:

- If nothing at all comes back, read returns **ETIMEDOUT**.
- If an intermediate router replies with an ICMP "destination unreachable" (host/network unreachable), the error is **EHOSTUNREACH** or ENETUNREACH. (The slide writes this as "ETIMEDOUT vs EDESTUNREACH".)
- To detect a crashed peer without sending data, use the **SO_KEEPALIVE** option (probe after 2 hours of idleness by default), or implement your own application-level timeout or heartbeat.

connect() to a host that is down (Minor 2 Q9 – two errors, and when):

- **ETIMEDOUT** – no response to the SYN at all. TCP retransmits the SYN (in 4.4BSD at 6, 24, ... seconds, giving up after about 75 s). Typical for a host on the same network whose ARP requests go unanswered, because the host is down or off the net.
- **EHOSTUNREACH** (or ENETUNREACH) – an intermediate router could not find a route to the host and sent an ICMP "destination unreachable" back (the host is on a different network and the router's table has no usable route).
- Contrast **ECONNREFUSED** – the host *is* up but no process is listening on that port; the host replies **RST** immediately.

6.4 Server host crashes and reboots

The server host crashes and comes back, losing all connection state. When the client's next data arrives, the rebooted host has no knowledge of the connection, and replies **RST**. The client's read returns **ECONNRESET**.

6.5 Server host shutdown

On a clean shutdown init sends **SIGTERM** to every process, waits ~5–20 s, then **SIGKILL**. A process that ignores SIGTERM is killed; the kernel closes its descriptors → FIN is sent, so the client sees the same sequence as 6.2.

---

7. I/O Multiplexing: select() (UNP Vol. 1, Ch. 6)

7.1 The need, and the five I/O models (Stevens 6.2)

A read on a blocking descriptor blocks until data arrives. A process that needs to watch *several* descriptors (stdin **and** a socket; many client sockets; a listening socket plus connected sockets) cannot do this with a plain blocking read: while it waits on one it misses the others.

Stevens lists five models:

1. **Blocking I/O** – default. Process sleeps until data is copied.
2. **Nonblocking I/O** – call returns EWOULDBLOCK/EAGAIN at once if not ready; you poll in a loop (wastes CPU).
3. **I/O multiplexing** – select/poll block *waiting for any of several descriptors* to become ready, then you do the actual read.
4. **Signal-driven I/O** – the kernel sends SIGIO when a descriptor is ready.
5. **Asynchronous I/O (aio_)** – the kernel does the whole operation (including the copy) and notifies you when finished.

Models 1–4 are all *synchronous* (the process blocks, at least during the copy of data from kernel to user). Only model 5 is truly asynchronous.

7.2 The select() function

```
#include <sys/select.h>
int select(int maxfdp1, fd_set *readset, fd_set *writeset,
           fd_set *exceptset, struct timeval *timeout);

void FD_ZERO(fd_set *fdset);          /* clear all bits           */
void FD_SET(int fd, fd_set *fdset);   /* turn on the bit for fd   */
void FD_CLR(int fd, fd_set *fdset);   /* turn off the bit for fd  */
int  FD_ISSET(int fd, fd_set *fdset); /* is the bit for fd on?    */
```

select() tells the kernel which descriptors to watch and for how long. It returns when:

- any descriptor in the **read set** is ready for reading,
- any in the **write set** is ready for writing,
- any in the **exception set** has an exceptional condition pending (for TCP: out-of-band data), or
- the **timeout** expires.

Arguments:

- `maxfdp1` – "max fd plus 1": the highest-numbered descriptor in any of the sets **plus one**. The kernel only examines descriptors 0 to maxfdp1–1. (Slide: `maxfd = max(stdin, connfd) + 1`.)
- Each of the three sets may be NULL if you don't care about that condition.
- `timeout`: NULL → wait forever. Zero timeval → return immediately (poll). Nonzero → wait at most that long (seconds + microseconds).

Return value: the **total number of ready descriptors** in all sets (> 0); **0** on timeout; **-1** on error (e.g. EINTR if a signal was caught).

Critical points:

- **The sets are value-result.** On return select has *modified* them: only the bits of descriptors that are ready remain set; all others are cleared. So you must **reset the sets (FD_ZERO + FD_SET) before every call**. This is the commonest select bug.
- After select returns, test with `FD_ISSET` to see which descriptors are ready.
- The number of descriptors is limited by **FD_SETSIZE** (typically 1024).
- `FD_SET(stdin, ...)` in the slide is shorthand; real code uses `fileno(stdin)` since FD_SET wants an int descriptor (0 for stdin) not a `FILE *`.

Why select() instead of read()/write() on each fd (Minor 3 Q6): a blocking read or write blocks on *one* descriptor and ignores the rest. select() blocks on *many descriptors at once* and returns as soon as *any* is ready, so the process is never stuck on a descriptor that has nothing to do while another needs attention. It can also time out, and the following read/write is then guaranteed not to block.

7.3 When is a descriptor ready? (slides: "Read / Write Descriptors are ready when…")

Ready for reading when:

- the number of bytes in the socket **receive buffer** ⩾ the **receive low-water mark** (SO_RCVLOWAT, default 1). A read will not block and returns the number of bytes available (> 0);
- the read half of the connection is closed (a FIN was received): a read returns **0 (EOF)**;
- it is a **listening socket** and the **completed connection queue is not empty**: accept won't block;
- a **socket error is pending**: read returns -1 with errno set.

Ready for writing when:

- the number of bytes of free space in the socket **send buffer** ⩾ the **send low-water mark** (SO_SNDLOWAT, default **2048** for TCP and UDP) and the socket is connected (or doesn't need a connection, as with UDP);
- the **write half is closed**: a write will generate **SIGPIPE** (and return EPIPE);
- a **socket error is pending**: write returns -1 with errno set;
- a **non-blocking connect** has completed or failed.

Exception condition: pending out-of-band data (TCP urgent mark).

Trick to remember: a non-blocking connect that *succeeds* makes the socket writable; one that fails makes it both readable and writable.

7.4 select() on stdin and a socket – str_cli (Stevens 6.4)

The old client was stop-and-wait: `fgets` (blocks on stdin) → `write` → `readline` (blocks on socket). While blocked on fgets it cannot see the server's FIN/RST (section 6.2). The new loop:

```
FD_ZERO(&rset);
for ( ; ; ) {
    FD_SET(fileno(fp), &rset);
    FD_SET(sockfd, &rset);
    maxfdp1 = max(fileno(fp), sockfd) + 1;
    select(maxfdp1, &rset, NULL, NULL, NULL);

    if (FD_ISSET(sockfd, &rset)) {           /* socket readable */
        if (Readline(sockfd, recvline, MAXLINE) == 0)
            err_quit("server terminated prematurely");
        Fputs(recvline, stdout);
    }
    if (FD_ISSET(fileno(fp), &rset)) {       /* input readable  */
        if (Fgets(sendline, MAXLINE, fp) == NULL)
            return;                          /* all done        */
        Writen(sockfd, sendline, strlen(sendline));
    }
}
```

The three outcomes: the peer sends data → socket readable; the peer sends FIN → socket readable and read returns 0; the peer sends RST → socket readable and read returns -1 with ECONNRESET.

7.5 A single-process TCP server using select() (Stevens 6.8)

Instead of fork-per-client, one process keeps an array `client[]` of connected descriptors (initialised to -1) and `maxi`, the highest index used:

1. Put listenfd and every valid client[i] in the read set; maxfd = highest fd. Call select.
2. If listenfd is readable: accept the new connection (it will not block), store connfd in the first free slot of client[], add it to the set, update maxfd and maxi.
3. For each client[i] whose descriptor is readable: read. If read returns 0 (EOF) → close it, remove it from the set, mark client[i] = -1. Else echo the data.

Advantages: no process-per-client overhead, no zombies, no SIGCHLD handling. Drawback: **denial-of-service** – if the server blocks in a read/write for one client (e.g. a malicious client that sends a partial line), every other client is stuck. The remedy is non-blocking I/O, threads, or fork. Also, only FD_SETSIZE descriptors can be monitored.

---

8. shutdown() and Half-Close (UNP Vol. 1, Ch. 6.6)

8.1 Motivation – the two limitations of close() (slide)

1. **close() only decrements the descriptor's reference count** and terminates the connection only when the count reaches 0. If a descriptor is shared with another process (after fork, or via dup), close() on one copy does *not* send a FIN.
2. **close() terminates both directions** of the connection. But TCP is **full-duplex** and allows a **half-close**: one side says "I am finished sending" while still being able to receive.

8.2 The function

```
#include <sys/socket.h>
int shutdown(int sockfd, int howto);   /* returns 0 on success, -1 on error
*/
```

- **SHUT_RD** – the read half is closed. No more data can be received; whatever is in the receive buffer is discarded, and any data that later arrives is acknowledged and silently discarded. A read returns 0. Nothing is sent to the peer for TCP.
- **SHUT_WR** – the write half is closed. Any data still in the send buffer is sent, **followed by TCP's normal connection-termination sequence (a FIN)**. The process can no longer write (a write gets **EPIPE / SIGPIPE**) but can still read. This is the **half-close**, and it is the most useful of the three.
- **SHUT_RDWR** – both halves are closed; equivalent to calling SHUT_RD then SHUT_WR.

(Older code uses the numeric values 0, 1, 2 for these.)

shutdown vs close, summarised:

- shutdown acts on the **connection** regardless of the reference count; close acts on the **descriptor** and acts on the connection only at count 0.
- shutdown can close **one direction**; close always closes both.
- shutdown does not free the descriptor; you still have to close() it afterwards.
- close() with SO_LINGER can wait for data to be delivered; shutdown(SHUT_WR) just queues the FIN after the pending data.

8.3 Why you need it – batch input (Stevens 6.5)

The select-based str_cli above has a bug when input comes from a file or pipe (batch mode) rather than a terminal. When fgets reaches **EOF on input** the client returns and exits – but the last requests may still be in flight (in the pipe to the server or in the network) and the replies not yet received. Simply closing the socket would throw away data still on its way back.

The fix: when the client reaches EOF on its input, call **`shutdown(sockfd, SHUT_WR)`** instead of returning. This sends a FIN after the data already queued, telling the server "no more requests", while the client keeps the read half open and continues the select loop to receive all remaining replies. When the server then closes (sends its own FIN), the client's read returns 0 and the client exits normally. Stevens adds a flag `stdineof`: once it is set, the client stops putting stdin in the read set. If the read returns 0 while stdineof is not set, the server terminated prematurely – that is an error; if stdineof is set it is normal.

8.4 Self-test (TCP / select / shutdown)

- Why does the server's parent call close(connfd) but not close(listenfd), and the child the reverse?
- Explain what happens if a SIGCHLD handler uses wait() once and five children exit together.
- What happens to a write after the peer's RST? After the peer's FIN?
- Which errors does read return after a server host crash? After crash and reboot?
- Why must fd_sets be reinitialised before each select() call? What does maxfdp1 mean?
- List the conditions that make a socket readable. Writable.
- What is half-close and which howto value gives it? Why does batch input need it?

---

9. Past-Paper Questions That Fall in This Range – Model Answers

These come from her Minor 1, 2 and 3 papers in the uploaded PDF.

Minor 1 – Persistence types. See section 1.2: process-persistent, kernel-persistent, filesystem-persistent, with an example of each.

Minor 1 – Semaphore initialised to 0 / 1. See section 1.7: zero → deadlock; one → mutual exclusion works but producer/consumer need counting semaphores to coordinate.

Minor 2 – Shared memory vs other IPC. Section 2.1. **Define a socket.** Section 3.1. **Port 5000 vs 34835.** Section 3.2 (host vs network byte order, 0x1388 vs 0x8813). **Purpose of mmap().** Section 2.3.

Minor 2 – Why can't normal string functions be used with socket data? Section 3.3 – they assume a terminating `'\0'`; socket data is a stream of bytes with arbitrary values; use memset/memcpy/memcmp (or bzero/bcopy/bcmp).

Minor 2 – Two errors from connect() when the server host is down. Section 6.3 (ETIMEDOUT and EHOSTUNREACH, with the reasons).

Minor 2 – bind with INADDR_ANY vs 10.5.0.63. Section 4.3.

Minor 3 – Child closes connfd but the parent does not. Section 4.4 (reference count 2 → 1, no FIN, connection stays open, fd leak).

Minor 3 – select() vs read()/write() advantage. Section 7.2.

Minor 3 – Two reasons to use SO_REUSEADDR (outside the shutdown boundary but she asked it).

1. To **restart a server** immediately. If the previous server (or its child, still serving a connection) left a connection in TIME_WAIT or established on that port, bind() fails with EADDRINUSE. SO_REUSEADDR lets a listening server start and bind to its well-known port even if connections using that port still exist.
2. To allow **multiple servers on the same port** provided they bind **different local IP addresses** (e.g. one per interface/virtual host). Without it, a wildcard bind conflicts with a specific bind to the same port. (A third use is multicast; a fourth is allowing the same IP+port for different protocols/binding ordering for UDP.) SO_REUSEADDR does **not** let two TCP servers bind the *same* IP and port simultaneously.

Minor 3 Q12 – UDP server not shown in netstat; client blocks forever in recvfrom. The server never called **bind()**, so there is no socket bound to port 5000 – the server's socket has no local port (the kernel gave it an ephemeral one on the first send) and the client's datagrams to port 5000 are dropped.

---

10. One-Page Cheat Sheet

- **Semaphore:** sem_open returns SEM_FAILED; O_CREAT needs mode + value; O_EXCL → EEXIST; sem_wait EINTR; trywait EAGAIN; post is async-signal-safe and never lost; unlink removes the name, destruction after last close.

- **mmap:** MAP_SHARED (changes visible and saved) vs MAP_PRIVATE (COW); returns MAP_FAILED; offset page-aligned; fd closable; shm_open object has size 0 → ftruncate; shm_unlink takes the name; SIGBUS beyond file, SIGSEGV outside mapping.
- **Byte order:** network = big-endian; htons/ntohs 16-bit, htonl/ntohl 32-bit; use memset/memcpy not str* for binary data.
- **Value-result args:** accept, recvfrom, getsockname, getpeername. Value args: bind, connect, sendto.
- **inet_aton:** returns 1 valid, 0 invalid. inet_ntoa: static buffer, by-value argument. Prefer inet_pton/ntop.
- **listen:** backlog; two queues (incomplete SYN_RCVD, complete ESTABLISHED); kernel finishes the handshake before accept.
- **close:** decrements refcount; FIN only at 0. **shutdown:** SHUT_RD / SHUT_WR (half-close, sends FIN) / SHUT_RDWR; ignores refcount.
- **Zombies:** SIGCHLD handler, loop `waitpid(-1, &stat, WNOHANG)`; accept can return EINTR → retry; ECONNABORTED → ignore.
- **Failure cases:** server process dies → FIN then RST on next write → SIGPIPE/EPIPE; host crash → ETIMEDOUT or EHOSTUNREACH; crash + reboot → ECONNRESET.
- **select:** maxfd+1; sets are value-result (reset each time); return = count ready, 0 timeout, -1 error; readable = data ≥ low-water, EOF, listening socket with complete connection, pending error; writable = space ≥ 2048, write half closed (SIGPIPE), pending error, non-blocking connect done.

---

11. Tricky and Complex Questions with Model Answers

These are the "what really happens" and "find the bug" style of question. Try each one on paper before reading the answer. Where a detail depends on the OS (Linux vs BSD), the answer says so.

11.1 Semaphores

Q1. Program A does `sem_open("/lock", O_CREAT, 0644, 1)`, then `sem_wait`, then is killed with SIGKILL before it posts. Later program B runs the identical `sem_open("/lock", O_CREAT, 0644, 1)` followed by `sem_wait`. What happens? How would you fix the design?

A named semaphore is kernel-persistent: when A died its reference was closed, but the semaphore was **not** removed, and its value is 0 (A had decremented it). B's `sem_open` with O_CREAT finds the semaphore already existing, so the **mode and value arguments are ignored** – the value stays 0 and B's `sem_wait` blocks forever. Unlike System V (which has SEM_UNDO), Posix semaphores are not automatically released when the holder dies.

Fixes: call `sem_unlink` at start-up before `sem_open` (or use O_CREAT|O_EXCL and on EEXIST unlink and retry); use `sem_timedwait` so a stuck lock is detected; or use a lock that the kernel releases on process death (fcntl record lock / flock) when crash-safety matters.

Q2. Parent: `static sem_t s; sem_init(&s, 1, 0); if (fork()==0) { sem_post(&s); exit(0); } sem_wait(&s);` Does the parent ever return from sem_wait?

No. `pshared = 1` only *permits* use between processes; the `sem_t` object itself must live in memory that is genuinely shared. After fork() the child has a **copy-on-write private copy** of the data segment, so the child's post increments *its own copy* and the parent's copy stays 0. The parent blocks forever.

Fix: allocate the `sem_t` in shared memory (`mmap` with MAP_SHARED|MAP_ANONYMOUS, or shm_open + ftruncate + mmap), then `sem_init(sem, 1, 0)` there – or use a **named** semaphore, whose descriptor-like handle refers to the same kernel object in both processes after fork.

Q3. Stevens' producer/consumer stores into `buff[i % MAXITEMS]` with each thread's own local counter i. You now run two producers and one consumer. What goes wrong?

The counting semaphores (nempty/nstored) correctly count *how many* slots are free/full, but they do not say *which* slot. Two producers, each with its own i, compute slots independently, so they overwrite each other's items and the consumer's order no longer matches. Fix: keep a shared `nput` index (and the shared item counter) in the shared structure and update it **inside the mutex**: `wait(nempty); wait(mutex); buff[nput] = v; nput = (nput+1) % MAXITEMS; post(mutex); post(nstored);`. The consumer similarly uses a shared `nget`. With exactly one producer and one consumer, local indices are enough and the mutex is not even required.

Q4. Two programs lock semaphores A and B. P1 does wait(A) then wait(B); P2 does wait(B) then wait(A). Why can this deadlock and how do you prevent it?

If P1 acquires A and P2 acquires B before either takes its second semaphore, each waits for the one the other holds – a circular wait. Prevent it by imposing a **global lock order** (everyone takes A before B), or by using `sem_trywait` for the second semaphore and

backing off (release the first and retry).

Q5. A thread tests `sem_getvalue(&s, &v); if (v > 0) sem_wait(&s);` to avoid blocking. What is wrong?

Check-then-act race: between reading the value and calling sem_wait another thread may decrement it to 0, and you block. Use `sem_trywait` (atomic test-and-decrement) which returns -1 with EAGAIN instead of blocking. Also remember sem_getvalue may report 0 or a negative count of waiters depending on the implementation.

Q6. What is the output ordering of this program, and does it depend on where stdout goes?

```
sem_t *s = sem_open("/s", O_CREAT, 0644, 0);
if (fork() == 0) { printf("child\n"); sem_post(s); exit(0); }
sem_wait(s);
printf("parent\n");
```

The parent cannot pass sem_wait until the child posts, and the child prints before posting, so on a **terminal** (line-buffered stdout) the output is always `child` then `parent`. The semaphore works across fork because a named semaphore handle is inherited. But if stdout is redirected to a file or pipe it is **fully buffered**: each process flushes its buffer only at exit(), so the order in the file depends on which process exits first, and can be reversed. The semaphore orders the *printf calls*, not the *flushes*. (Add `fflush(stdout)` before the post to fix it.)

Q7. Why must sem_wait be called in a loop that checks EINTR?

If a signal handler runs while the thread is blocked, sem_wait returns -1 with errno = EINTR *without* having decremented the semaphore. Code that treats this as success would enter the critical section without the lock. Use `while (sem_wait(&s) == -1 && errno == EINTR) continue;`.

11.2 Shared memory and mmap

Q8. A 100-byte file is mapped with `mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_SHARED, fd, 0)` on a system with 4096-byte pages. What happens when you touch byte 99? Byte 200? Byte 5000? Byte 9000? Does the file grow?

- Byte 99: valid file data.
- Byte 200: inside the last (and only) page that contains file data. Reads return zeros; writes are allowed but **never saved** to the file.

- Byte 5000: the page is entirely beyond the end of the file → **SIGBUS**.
- Byte 9000: beyond the 8192-byte mapping → **SIGSEGV**.
- The file does **not** grow: mapping never extends a file. To make it bigger use `ftruncate()` (or lseek past the end and write one byte) *before* accessing those pages.

Q9. Two processes each execute `for (i = 0; i < 1000000; i++) (*counter)++;` on an int in a MAP_SHARED region. Is the final value 2,000,000? Why/why not?

Not reliably. `(*counter)++` compiles to load → add → store. If both processes load the same value before either stores, one increment is lost. The result is at most 2,000,000 and typically far less. The kernel provides no synchronisation for shared memory; protect the increment with a semaphore initialised to 1 (or a process-shared mutex, or an atomic instruction).

Q10. A file is opened `O_RDONLY` and mapped with `PROT_READ|PROT_WRITE`. Does `MAP_SHARED` work? Does `MAP_PRIVATE`?

`MAP_SHARED` fails with **EACCES** – writes would have to go back to a file you opened read-only. `MAP_PRIVATE` **succeeds**, because writes only modify a private copy-on-write copy and are never written back; you simply can't save your changes.

Q11. After `fork()`, a parent's `malloc`'d array, a global variable and a `MAP_SHARED` mapping are all modified by the child. Which changes does the parent see?

Only the **MAP_SHARED** mapping. The heap array and globals are copy-on-write private copies after fork. A `MAP_PRIVATE` mapping behaves like the heap: private. So to share data between related processes created by fork, you must create the MAP_SHARED mapping *before* the fork.

Q12. A process calls `shm_unlink("/data")` while two processes still have it mapped. What happens?

The *name* is removed immediately, so a new `shm_open("/data", O_RDWR, 0)` fails with ENOENT (and an `O_CREAT` creates a **brand-new, different** object). The processes that already have it mapped keep working normally; the object itself is destroyed only when the last one unmaps it.

Q13. You need the last 10 bytes of a 1 MB file. Why does `mmap(NULL, 10, PROT_READ, MAP_SHARED, fd, 1048566)` fail, and what do you do?

The offset must be a multiple of the page size (4096); 1048566 is not, so mmap fails with **EINVAL**. Compute `pa_offset = offset & ~(pagesize-1)`, map from `pa_offset` with length `length + (offset - pa_offset)`, then add `(offset - pa_offset)` to the

returned pointer.

Q14. For transferring 1 GB between two processes on one host, compare a pipe with shared memory.

Pipe: every chunk (at most the pipe buffer size) is copied user → kernel, then kernel → user, requiring a write() and a read() system call per chunk and a context switch whenever the pipe fills or empties. Kernel provides the blocking/synchronisation. Shared memory: after one-time setup (shm_open, ftruncate, mmap) there are no copies and no system calls for the data; the processes read and write the same physical pages. The cost is that **you** must synchronise (semaphores for full/empty) and must define your own message boundaries and a protocol for flow control.

11.3 Sockets foundations

Q15. `struct mesg { unsigned int seq_no; unsigned short reserved; unsigned char data[16]; };` What is `sizeof(struct mesg)` and why might a client and server disagree on it?

The members add to 4 + 2 + 16 = 22 bytes, but the struct's alignment is 4 (its largest member), so the compiler pads the total to a multiple of 4: **sizeof is 24** on typical ABIs, with 2 bytes of trailing padding (offsets: seq_no 0, reserved 4, data 6). Consequences in a TCP program: (a) a sender that writes sizeof(struct) = 24 bytes and a receiver that expects 22 (different compiler options, `#pragma pack(1)`, another language) will desynchronise – TCP has **no message boundaries**, so the extra 2 bytes become the start of the *next* message; (b) the padding bytes are uninitialised memory sent on the wire; (c) `seq_no` and `reserved` also need htonl/htons if the hosts may differ in endianness. Fix: pack the struct (`__attribute__((packed))` / `#pragma pack(1)`), or better, serialise field by field in network byte order, and always read with a loop until exactly the expected number of bytes has arrived.

Q16. On a little-endian machine, what are `htons(0x1234)`, `htonl(1)`, and the value printed for `sin_port` if a server bound to port 8080 and you print it without `ntohs`?

- `htons(0x1234)` = **0x3412** (bytes swapped).
- `htonl(1)` = **0x01000000 = 16777216**.
- 8080 = 0x1F90; stored big-endian as bytes 1F 90; read back as a little-endian host short that is **0x901F = 36895**.
- On a big-endian host all of these functions are no-ops. The macros exist so the *same source* works on both.

Q17. What does `printf("%s %s\n", inet_ntoa(a), inet_ntoa(b));` print?

`inet_ntoa` returns a pointer to a **single static buffer** that each call overwrites. Both arguments are evaluated before printf runs, so both `%s` print the *same* (the last-evaluated) address. It also makes inet_ntoa unsafe in multithreaded code. Use `inet_ntop` with your own buffers.

Q18. What do `inet_aton("127.1")`, `inet_aton("1.2.3")`, `inet_aton("0x7f.0.0.1")` and `inet_aton("010.1.1.1")` return, and what does `inet_pton(AF_INET, ...)` do with them?

inet_aton accepts the old BSD forms: "127.1" → 127.0.0.1 (the last number fills all the remaining bytes); "1.2.3" → 1.2.0.3; "0x7f.0.0.1" → 127.0.0.1 (hex allowed); "010.1.1.1" → **8**.1.1.1 (a leading 0 means **octal**). All return 1 (valid). `inet_pton(AF_INET, ...)` accepts only the strict decimal dotted-quad `a.b.c.d` and rejects all four (returns 0). Another trap: `inet_addr("255.255.255.255")` returns INADDR_NONE, indistinguishable from an error.

Q19. Why is the length argument of `accept()` a pointer and why must you re-initialise it inside the loop?

It is a **value-result** argument: on entry it says how large your address buffer is, on return how many bytes the kernel actually stored. If you leave the old (possibly different) result in it, or an uninitialised value, the kernel may truncate the address or fail with EINVAL. So `len = sizeof(cliaddr);` belongs inside the `for(;;)` before each accept. bind/connect/sendto take the length by value because they only pass it *into* the kernel.

11.4 TCP, concurrency and failures

Q20. The server calls `listen(fd, 5)` and then sleeps for 60 seconds before the first accept. A client connects and sends 100 bytes. What does the client see? What does netstat show? What if 20 clients try?

The client's `connect()` **succeeds immediately** – the kernel completes the three-way handshake without involving the application – and its `write` succeeds because the data is buffered in the server's receive buffer of the not-yet-accepted connection. netstat shows ESTABLISHED on both ends. Once the completed-connection queue is full, further handshakes do not complete: the server TCP ignores the extra SYN/ACK, so those clients' connect() calls just keep retransmitting SYN and appear to hang for a long time (they are not refused). When the server finally accepts, the buffered data is already there.

Q21. A client connects and immediately closes. The server, unaware, calls write() twice with a delay between them. Describe what happens at each write.

First write: the client TCP has already sent FIN and the socket is closed, so it **replies with RST** to the data; the first write itself returns success (the data just went into the send buffer). By the time of the second write, the RST has been received, so the second write generates **SIGPIPE** (default: terminate process) – or returns -1/EPIPE if SIGPIPE is ignored or handled. The server therefore needs `signal(SIGPIPE, SIG_IGN)` and must check write()'s return value.

Q22. Five clients terminate at almost the same moment and the server's SIGCHLD handler calls `wait()` once. How many zombies remain? Why?

Some remain – typically most of them. Signals are **not queued**: if SIGCHLD is already pending, additional SIGCHLDs arriving before the handler runs are merged into one. The handler runs once (or twice), each `wait()` reaps a single child, and the rest stay as zombies (`ps` shows state Z / `<defunct>`). The exact number depends on timing, which is the point. The fix is the loop `while (waitpid(-1, &stat, WNOHANG) > 0);` .

Q23. After adding a SIGCHLD handler, the server dies with "accept error: Interrupted system call". Why, and what are two fixes?

SIGCHLD arrived while the parent was blocked in accept(), a **slow system call**. When a handler returns, such a call is interrupted and returns -1 with errno = EINTR (unless the signal was installed with SA_RESTART). The server code treated every accept failure as fatal. Fixes: (1) `if (errno == EINTR) continue;` after a failed accept (and treat ECONNABORTED the same way); (2) install the handler with `sigaction` and `SA_RESTART` . Portable code does (1) regardless.

Q24. The server's child process forgets to close `listenfd` . What are the consequences? And if the parent forgets to close `connfd` ?

Child forgets listenfd: the listening socket's reference count is higher than needed. If the parent dies or is restarted, the listening socket stays alive in the children (the port remains bound, in LISTEN state), so a restarted parent fails to bind with EADDRINUSE until all children exit. It also wastes a descriptor per child. Parent forgets connfd: (1) the parent leaks one descriptor per client until accept fails with EMFILE; (2) when the child closes its copy, the reference count only drops from 2 to 1, so **no FIN is sent** and the client never sees EOF – the connection stays open until the parent exits.

Q25. `netstat` on the server shows hundreds of connections in CLOSE_WAIT, and on the clients FIN_WAIT_2. What is the bug?

The clients closed first (FIN sent, ACKed → client FIN_WAIT_2). The server TCP received the FIN and is in CLOSE_WAIT, waiting for the **server application** to call close(). So the server application is not closing its descriptors after read returns 0 (EOF) – a descriptor leak (often a missing close(connfd) in the parent, as in Q24). CLOSE_WAIT never times out by itself.

Q26. Why does restarting a server immediately fail with "Address already in use" even though the old process is gone? Why does TIME_WAIT exist?

The side that performs the **active close** (sends the first FIN) keeps the connection in TIME_WAIT for 2×MSL (typically 1–4 minutes). If the server (or its child) closed first, a connection with the server's well-known port stays in TIME_WAIT, so a plain bind() to that port fails with EADDRINUSE. `SO_REUSEADDR` set before bind allows it. TIME_WAIT exists for two reasons: (1) to be able to **retransmit the final ACK** if it was lost (otherwise the peer's retransmitted FIN would get an RST and the close would not be reliable); (2) to let **old duplicate segments** from that connection die out in the network so they are not mistaken for data of a new incarnation of the same 5-tuple.

Q27. A client is blocked in `read()` waiting for a reply and the server host's power is pulled. How long until the client notices? What changes if the client was blocked in write/retransmitting?

If the client is **only reading** and never sends anything, it **never notices** – no packets are exchanged, so nothing times out. With `SO_KEEPALIVE` on, a probe is sent after the connection is idle for 2 hours (Linux default 7200 s), then up to 9 probes at 75 s intervals, i.e. about 2 h 11 min, after which read returns ETIMEDOUT. If the client had been *sending* data, TCP retransmits with exponential backoff for about 9 minutes before reporting ETIMEDOUT (or EHOSTUNREACH if a router returns ICMP unreachable). To detect it faster, use an application-level heartbeat or a select() timeout.

Q28. Rank by how quickly the client learns of the failure: connecting to (a) a live host with no listener on the port, (b) a host that is powered off on the same LAN, (c) a host behind a router that has no route to it.

(a) **Fastest:** the host answers the SYN with RST → ECONNREFUSED immediately (a hard error). (c) A router returns ICMP "destination unreachable" → EHOSTUNREACH/ENETUNREACH (often quickly; Linux reports it as soon as the ICMP arrives). (b) **Slowest:** nobody answers at all, ARP fails / SYN retransmissions go unanswered → ETIMEDOUT after about 75 s (BSD) to ~2 minutes (Linux).

11.5 select() and shutdown()

Q29. This loop never notices keyboard input after the first socket message. Find the bug.

```
FD_ZERO(&rset); FD_SET(0, &rset); FD_SET(sockfd, &rset);
for ( ; ; ) {
    select(sockfd + 1, &rset, NULL, NULL, NULL);
    if (FD_ISSET(sockfd, &rset)) handle_socket();
    if (FD_ISSET(0, &rset))      handle_stdin();
}
```

The fd_set is a **value-result** argument. When select returns because only the socket is ready, it *clears* the bit for descriptor 0. The sets are initialised once, outside the loop, so on the next call descriptor 0 is no longer being watched – and stdin is ignored forever. Move `FD_ZERO` and both `FD_SET` calls **inside** the loop (or keep a master copy and assign `rset = allset` each iteration, as Stevens does in the server).

Q30. A client uses descriptors 0 (stdin) and 4 (socket) and calls `select(4, ...)`. What happens?

The first argument must be the highest-numbered descriptor **plus one**, i.e. 5. With 4, the kernel examines only descriptors 0–3, so the socket (4) is never checked and the program hangs (or only ever sees stdin). The `maxfdp1` name stands for "max fd plus 1".

Q31. select() reports the listening socket readable but the following accept() blocks forever. How is that possible?

A race: the connection was complete when select looked, but the client sent RST (aborted) before accept ran, and the kernel removed it from the queue (Berkeley behaviour). The queue is now empty, so a blocking accept sleeps although select said "readable". Stevens' remedy: set the **listening socket non-blocking** when using select, and ignore EWOULDBLOCK, ECONNABORTED, EPROTO and EINTR from accept.

Q32. After `fork()`, the child calls `close(connfd)`; another child calls `shutdown(connfd, SHUT_WR)`. When does the peer see EOF in each case?

With `close`, the reference count only goes from 2 to 1 – no FIN is sent, so the peer sees nothing until the *last* holder closes. With `shutdown(SHUT_WR)`, the FIN is sent **immediately**, whatever the reference count, because shutdown acts on the connection, not on the descriptor. (The data already queued is delivered first, then the FIN.) The descriptor remains open and must still be closed.

Q33. A client in batch mode writes 1000 requests and calls `close(sockfd)` immediately. Many replies are still on their way. What is lost, and what is the correct approach?

close() terminates **both directions**: the process can no longer read, and any reply that arrives for a fully closed socket is discarded by TCP and answered with **RST**. The client loses the outstanding replies, and the server's later writes may fail with ECONNRESET/EPIPE (and SIGPIPE). Correct: when input reaches EOF, call `shutdown(sockfd, SHUT_WR)`. The FIN follows the last request, the server reads EOF *after* processing all requests, sends all replies and closes; the client, still reading, drains every reply and exits when its read returns 0.

Q34. What does the peer observe when a TCP host calls `shutdown(fd, SHUT_RD)`? When it calls `shutdown(fd, SHUT_WR)`?

SHUT_RD: **nothing** is sent to the peer for TCP. The peer's writes keep succeeding; the data is acknowledged by the local TCP and silently discarded. (So the peer cannot tell the read side is closed; contrast with UDP semantics.) SHUT_WR: a **FIN** is sent after the queued data, so the peer's read returns 0 (EOF) once it has consumed the data, but the peer can still write to us, and our process can still read.

Q35. After `shutdown(fd, SHUT_WR)`: can the process still read? What does write do? Is the descriptor released?

Yes, it can still read until the peer's FIN arrives (and then read returns 0). A write fails with **EPIPE** and generates **SIGPIPE**. The descriptor is not released: shutdown never frees it, so `close(fd)` must still be called.

Q36. On Linux, `select()` is called with `struct timeval t = {5, 0};` in a loop, and the loop does not reset t. What happens?

Linux **modifies the timeout** to the time remaining. If select returned after 3 seconds because a descriptor became ready, t now holds {2, 0}; the next call waits only 2 seconds, and once it reaches {0, 0} select just polls and returns immediately, causing a busy loop. Other systems leave it unchanged, so portable code **re-initialises the timeval before every call**, together with the descriptor sets.

Q37. Design: a server must handle 5,000 simultaneous idle connections. Compare fork-per-client, one process with select(), and one process with poll().

- **fork-per-client:** simple model and good isolation, but 5,000 processes use a lot of memory and context-switch overhead; must handle zombies; sharing state between clients needs IPC.

- **select() single process:** no per-client process, no zombies; but limited to **FD_SETSIZE (1024)** descriptors, and every call copies all three sets into the kernel and scans them (O(n) per call); the sets must be rebuilt each time; one slow handler blocks all clients (Stevens' denial-of-service example), so I/O must be non-blocking.
- **poll():** no fixed FD_SETSIZE limit (an array of struct pollfd with `events` and `revents`, so no need to rebuild the input each call); still O(n) scanning per call. Beyond a few thousand connections platform-specific mechanisms (epoll/kqueue) scale better.

Q38. Which descriptor conditions make `select()` report "readable" even though `read()` will not return data?

(1) EOF – the peer's FIN arrived and read returns **0**. (2) A **pending socket error** (e.g. RST received) – read returns -1 with ECONNRESET/ETIMEDOUT etc. (3) A **listening socket** with a completed connection – you call accept, not read. Also remember the write-side surprises: a socket whose write half is closed is "writable" (write returns immediately with SIGPIPE/EPIPE), and a failed non-blocking connect makes the socket both readable and writable.

11.6 Rapid-fire "what happens when…" list

- `sem_open` without O_CREAT, semaphore missing → SEM_FAILED, ENOENT.
- `sem_open(O_CREAT|O_EXCL)` on an existing semaphore → SEM_FAILED, EEXIST.
- `sem_trywait` on value 0 → -1, EAGAIN.
- `sem_unlink` while processes still have it open → name gone now, object destroyed at last close.
- `mmap` returns → compare against MAP_FAILED, never NULL.
- Write to a mapped region opened O_RDONLY with MAP_SHARED, PROT_WRITE → EACCES at mmap time.
- Touch page beyond EOF → SIGBUS; outside mapping → SIGSEGV.
- `shm_open` new object then mmap without ftruncate → object size 0 → SIGBUS on access.
- `bind` to a port in use (and no SO_REUSEADDR) → EADDRINUSE.
- `connect` to a live host, no listener → ECONNREFUSED. Host down same LAN → ETIMEDOUT. Router has no route → EHOSTUNREACH.
- `accept` interrupted by signal → EINTR (retry). Client reset before accept → ECONNABORTED (retry).

- `read` returns 0 → peer sent FIN (EOF). `read` returns -1/ECONNRESET → peer sent RST.
- First `write` after peer's RST → SIGPIPE (EPIPE if ignored).
- `close` with another process holding the descriptor → no FIN. `shutdown(SHUT_WR)` → FIN regardless.
- `select` returns 0 → timeout; -1 with EINTR → signal; > 0 → number of ready descriptors (not number of events).